Feedback Intelligence Platform: Project Overview

October 13, 2026

What the program is

The Feedback Intelligence Platform reads public GitHub issues from a few popular open-source products every day, works out what users are struggling with, and publishes an always-current dashboard for product teams. It runs unattended, costs $0, and stays that way.

The problem. Popular software gets more feedback than any team can read; VS Code alone receives hundreds of issues a week. A product manager wants to know what users complain about most, what is suddenly spiking, whether fixes are getting faster, and which reports need attention today. This program does the reading and sorting so they don't have to.

What it provides, every day:

- **A category** for each issue: bug, feature request, question, docs, or performance.
- **A severity score** (low, medium, high, critical) with a one-line reason.
- **A plain-English summary** of each long issue.
- **Emerging topics:** groups of similar issues that are growing fast.
- **Trends:** volume, category mix, and time-to-close per product over time.
- **A weekly digest:** a short AI-written briefing of the top themes.
- **Privacy protection:** emails, names, tokens and IP addresses are redacted before anything reaches the AI or the public site.

Who uses it and how:

Role	Who	How they interact
Viewer	Product managers, engineers, interviewers	Open a public URL. Browse Overview, Trends, Emerging topics, Issue explorer, Weekly digest and Pipeline health pages. No login.
Admin / operator	You	Change behavior through pull requests (config, prompts, SQL). Trigger manual runs and backfills from GitHub Actions. Respond to alert issues. Develop locally with Docker.

Out of scope: user accounts, writing back to GitHub, real-time streaming, a live chat box (it would need a paid server or exposed key), and storing author identities on the public site.

End-to-end process

The work happens in two layers: the SDLC phases you follow to build it once, and the daily pipeline it runs on its own forever after.

Part A: Building it (SDLC phases)

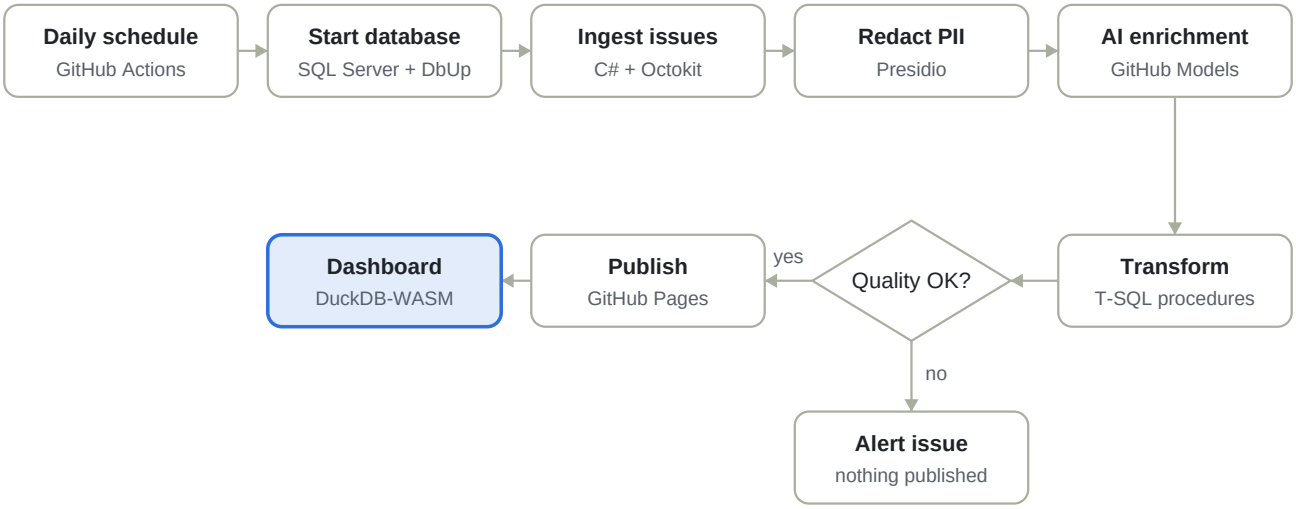
1. **Requirements.** Write user stories with acceptance criteria and non-functional targets (data under 24 hours old, $0 cost, no PII published). Track them as GitHub Issues on a GitHub Project board.
2. **Design.** Draw the architecture, define the database schema and data contracts, write a short threat and privacy review, and record key choices as Architecture Decision Records (ADRs).

3. **Development standards.** Set up branching, commit conventions, a PR template with an AI-disclosure section, formatting rules, and the AI-assisted development guide.
4. **Walking skeleton.** Build the thinnest possible end-to-end path: fetch one issue, store it in SQL Server, publish one number to the dashboard, all through CI. This proves the plumbing before features are added.
5. **Feature iterations.** Add the pieces one PR at a time: incremental ingestion, PII redaction, T-SQL transforms, AI enrichment, topic clustering, dashboard pages. Each PR ships with tests.
6. **Testing.** Unit tests for C# classes, integration tests against a real SQL Server container, SQL expected-output tests, data-quality checks, and an accuracy evaluation for the AI prompts.
7. **Deployment.** GitHub Actions runs CI on every PR and deploys the dashboard to GitHub Pages on merge. The daily schedule turns on.
8. **Operations.** Define freshness and success targets, alert rules, a runbook, and a postmortem template. Run a simulated incident and write it up.
9. **Feedback loop.** Use dashboard usage and pipeline metrics to choose the next improvements.

Part B: Running it (the daily pipeline)

1. **Trigger.** At 06:00 UTC, a scheduled GitHub Actions workflow starts.
2. **Start the database.** A SQL Server Express container starts; DbUp applies any new schema migrations.
3. **Restore history.** Prior data, stored as Parquet files in the repo, is loaded into SQL Server.
4. **Ingest.** The C# app fetches only issues created or changed since the last run.
5. **Redact PII.** Each issue's text passes through Microsoft Presidio; personal data is replaced with placeholders.
6. **Enrich with AI.** New or changed issues are sent to GitHub Models for category, severity and summary. Results are cached by content hash so no text is sent twice.
7. **Transform.** T-SQL stored procedures move data from raw to cleaned to reporting tables, using MERGE upserts and window functions.
8. **Check quality.** SQL checks verify row counts, nulls, schema and freshness. On failure the run stops and opens a GitHub issue as an alert; nothing bad is published.
9. **Publish.** Reporting tables are exported as Parquet to GitHub Pages; updated history and run metrics are committed back to the repo.
10. **View.** Anyone opens the dashboard URL; the browser queries the data with DuckDB-WASM.

Each daily run publishes only data that passes quality checks



The run flows left to right, then back along the bottom row; a failed quality check opens an alert instead of publishing.

Technologies

Every tool below is free with no trial or subscription. Each row says what the tool is, what it does in this program, and why it was chosen.

Application code

Technology	What it is	Used for here	Goal
C# / .NET 8	Microsoft's object-oriented language and runtime	The pipeline app: ingestion, redaction calls, AI calls, export	Show strong OOP and a JD-listed language
Octokit.NET	Official C# client for the GitHub API	Fetch issues incrementally with paging and rate-limit handling	Reliable, typed access to the data source
OpenAI .NET SDK	Official C# client for OpenAI-style APIs	Call GitHub Models for classification and summaries	Same code works against Azure OpenAI by changing the endpoint
Parquet.Net	C# library for the Parquet columnar file format	Store history and export reporting data	Compact, fast files the browser can query
Microsoft.Extensions (DI, Logging, Configuration)	.NET's built-in dependency injection, logging and config	Wire components together behind interfaces; read config.yaml	Testable, swappable design; config-as-code

Data and SQL

Technology	What it is	Used for here	Goal
SQL Server Express	Free production edition of SQL Server (10 GB limit)	The warehouse during each pipeline run	Real SQL Server in production, legally free

Technology	What it is	Used for here	Goal
SQL Server Developer	Free full-feature edition licensed for dev/test	Local development and exploration	Full features on your laptop; correct licensing
T-SQL	SQL Server's SQL dialect	Stored procedures, MERGE upserts, window functions, views, data-quality checks	Demonstrate deep SQL skill
DbUp	C# library that runs versioned .sql migration scripts	Apply schema changes in order on every run	Controlled schema evolution
SSMS / VS Code SQL extension	Tools to query and inspect SQL Server	Query tuning, execution plans, live demos	Show indexing and performance work

AI and privacy

Technology	What it is	Used for here	Goal
GitHub Models	Free, rate-limited API to OpenAI and other models	Category, severity, summaries, weekly digest	AI integration at $0
Microsoft Presidio	Microsoft's open-source PII detection, run as a Docker REST service	Redact emails, names, tokens, IPs before AI or publishing	Privacy and compliance by design
Prompt evaluation set	~100 hand-labeled issues plus a C# accuracy test	Block prompt changes that lower accuracy	Treat AI output as tested code
GitHub Copilot Free	AI coding assistant	Generate boilerplate, SQL, tests and docstrings, all reviewed	Disciplined AI-assisted development

Testing

Technology	What it is	Used for here	Goal
xUnit	Standard .NET test framework	Unit tests for every class	Reliable, testable code
Testcontainers for .NET	Library that starts real services in Docker for tests	Integration tests against real SQL Server	Test against the real engine, not mocks
SQL quality checks	Assertion queries that must return zero bad rows	Stop the pipeline on bad data	Never publish wrong numbers

Automation and operations

Technology	What it is	Used for here	Goal
Git + GitHub	Version control and hosting	Code, PRs, reviews, issues, project board	Visible engineering history
GitHub Actions	GitHub's CI/CD and scheduler	PR checks, daily pipeline, deployment, alerts	Free production runtime
GitHub Pages	Free static website hosting	Host the public dashboard	Permanent public URL at $0
GitHub Issues (as alerts)	Issue tracker	Auto-opened on pipeline failure	Real alerting and incident log
Docker + Docker Compose	Containers and multi-container setup	Run SQL Server, Presidio and Grafana locally with one command	Reproducible environment, offline demos

Technology	What it is	Used for here	Goal
OpenTelemetry + Grafana	Telemetry standard and dashboards	Local metrics and traces during development	Monitoring skills; Azure Monitor accepts OpenTelemetry
Bicep	Azure's infrastructure-as-code language	Templates validated in CI, never deployed	"Deploys to Azure by config" story

Dashboard

Technology	What it is	Used for here	Goal
HTML + JavaScript	Standard web page code	Dashboard pages and filters	No server needed
DuckDB-WASM	A SQL engine that runs inside the browser	Query exported Parquet as users filter	Fast analytics with no backend
Chart.js	Free charting library	Trend and topic charts	Clear visuals

Staying at $0 and matching the job description

Nothing in the stack has a trial clock or a credit balance, so the live demo keeps working until you choose to stop it.

Free-tier rules to respect:

- **GitHub Models** is free but rate-limited, about 15 requests a minute and 150 a day for lower-tier models on a free account, and GitHub can change these limits. Caching and a daily batch keep usage well under this.
- **Public repo** keeps GitHub Actions minutes free.
- **Scheduled workflows** pause after about 60 days with no repo activity. The pipeline commits its own outputs, and a check before interviews takes one click.
- **No payment method on GitHub.** A user reported an unexpected bill under Copilot's newer usage-based billing; with no card on file and paid usage left off, nothing can be charged.
- **SQL Server licensing:** Express in the pipeline (production), Developer only on your laptop.
- **Backups that never expire:** the repo itself, docker compose up for offline demos, and a recorded walkthrough video.

Job-description coverage:

JD item	Where it shows
C#, OOP, data structures	Interface-based C# app; hashing for dedup and caching; heap for top-K topics
SQL, SQL scripting, SQL Server	T-SQL transforms, MERGE, window functions, indexing and query-plan write-up, partitioning
AI/ML, OpenAI	GitHub Models enrichment, prompt evaluation set, weekly digest
Responsible AI-assisted development	Copilot use logged in AI_LOG.md, PR disclosure, reviewed output
Testing and reliable deployment	xUnit, Testcontainers, SQL quality checks, CI gates
Security, privacy, compliance	Presidio redaction, licensing ADR, no secrets in code
Monitoring, logging, alerting	Pipeline Health page, run metrics, auto-opened alert issues

JD item	Where it shows
Telemetry and usage analytics	Run metrics and dashboard usage driving the backlog
Git and DevOps	PR workflow, GitHub Actions, migrations in the pipeline
Azure	Bicep templates and an OpenAI-compatible client ready to point at Azure
Collaboration and leading through influence	Peer PR reviews, ADRs recording disagreements, a Copilot best-practices guide for others

Not covered for free: Fabric (no permanent free tier). The T-SQL work transfers directly to a Fabric warehouse.

Sources

GitHub Models free-tier limits: https://aireiter.com/blog/best-free-ai-api

GitHub Models billing: https://docs.github.com/en/enterprise-cloud@latest/billing/concepts/product-billing/github-models

Copilot Free billing discussion: https://github.com/orgs/community/discussions/207246